

INTELIGENCIA ARTIFICIAL

---

# Guía de Laboratorio

## Búsqueda Informada con Pac-Man

---

Algoritmo A\*

Heurísticas

Distancia Manhattan y Euclidiana

Diseño de heurísticas

Estudiante(s): \_\_\_\_\_

Grupo: \_\_\_\_\_

Fecha: \_\_\_\_\_

## Introducción

Los algoritmos de búsqueda constituyen una herramienta fundamental para resolver problemas en Inteligencia Artificial. En los algoritmos de búsqueda no informada, la exploración del espacio de estados se realiza sin utilizar información adicional acerca de la ubicación del objetivo.

La **búsqueda informada** introduce una función heurística que permite estimar qué tan conveniente es explorar un determinado estado.

En este laboratorio se utilizará el entorno de **Pac-Man** como espacio experimental para estudiar el algoritmo  $A^*$  y diferentes funciones heurísticas.

El laboratorio está inspirado en los proyectos académicos de Pac-Man utilizados para la enseñanza de Inteligencia Artificial.

Página de referencia:

<https://stanford-cs221.github.io/spring2023/assignments/pacman/index.html>

El propósito principal no es simplemente lograr que Pac-Man llegue al objetivo, sino analizar:

- cómo se representa un problema de búsqueda;
- cómo funciona  $A^*$ ;
- cómo una heurística modifica la exploración;
- cuántos estados deben expandirse;
- cómo diseñar mejores heurísticas.

## Objetivos

### Objetivo

Implementar y analizar algoritmos de búsqueda informada utilizando el entorno Pac-Man, evaluando el efecto de diferentes funciones heurísticas sobre la cantidad de estados explorados y el costo de la solución.

Al finalizar el laboratorio, el estudiante estará en capacidad de:

- identificar los componentes de un problema de búsqueda;
- diferenciar búsqueda no informada e informada;
- implementar el algoritmo  $A^*$ ;
- utilizar funciones heurísticas;
- comparar heurísticas Manhattan y Euclidiana;
- analizar admisibilidad y consistencia;
- diseñar una heurística para un problema de búsqueda;
- evaluar experimentalmente la eficiencia de un algoritmo.

## El problema de búsqueda

Un problema de búsqueda puede representarse mediante los siguientes componentes:

$$P = (S, A, T, s_0, G, C)$$

donde:

- $S$  representa el conjunto de estados;
- $A$  representa las acciones disponibles;
- $T$  representa la función de transición;
- $s_0$  representa el estado inicial;
- $G$  representa la prueba de objetivo;
- $C$  representa el costo de las acciones.

En el entorno Pac-Man, un estado sencillo puede corresponder a una posición:

$$s = (x, y)$$

y las acciones disponibles son:

$$A = \{North, South, East, West\}$$

siempre que el movimiento correspondiente no conduzca a una pared.

## Búsqueda informada

La diferencia principal entre búsqueda no informada y búsqueda informada consiste en la incorporación de conocimiento acerca del problema.

Una función heurística se representa mediante:

$$h(n)$$

y estima el costo necesario para desplazarse desde un estado  $n$  hasta un estado objetivo.

En A\* se utiliza:

$$f(n) = g(n) + h(n)$$

donde:

$$g(n) = \text{costo acumulado desde el estado inicial hasta } n$$

y

$$h(n) = \text{estimación del costo restante hasta el objetivo.}$$

Por lo tanto:

$$\boxed{f(n) = g(n) + h(n)}$$

A\* selecciona para expansión el estado que presenta el menor valor de  $f(n)$ .

## Reconocimiento del proyecto Pac-Man

El proyecto se encuentra organizado mediante diferentes archivos Python.

Los archivos más importantes para este laboratorio son:

| Archivo                      | Función                                                                 |
|------------------------------|-------------------------------------------------------------------------|
| <code>pacman.py</code>       | Control principal del entorno Pac-Man.                                  |
| <code>search.py</code>       | Implementación de algoritmos de búsqueda.                               |
| <code>searchAgents.py</code> | Definición de problemas de búsqueda, agentes y heurísticas.             |
| <code>util.py</code>         | Estructuras de datos auxiliares como pilas, colas y colas de prioridad. |
| <code>layout.py</code>       | Lectura y representación de los laberintos.                             |

En particular, el archivo:

`search.py`

contiene la clase:

```
1 class SearchProblem:
2
3     def getStartState(self):
4         ...
5
6     def isGoalState(self, state):
7         ...
8
9     def getSuccessors(self, state):
10        ...
11
12    def getCostOfActions(self, actions):
13        ...
```

Esta estructura define la interfaz general utilizada por los algoritmos de búsqueda.

## Actividad 1. Exploración del entorno

### Actividad 1 – Ejecutando Pac-Man

Desde una terminal ubíquese en el directorio del proyecto.

Ejecute:

```
python pacman.py
```

Utilice las teclas de dirección para explorar manualmente el laberinto.

También puede utilizar:

W, A, S, D

para controlar a Pac-Man.

Observe:

- posición inicial;
- paredes;
- alimentos;
- movimientos permitidos;
- estados alcanzables.

### Pregunta de análisis

Identifique los componentes del problema de búsqueda dentro del entorno Pac-Man.

| Elemento        | Descripción |
|-----------------|-------------|
| Estado          |             |
| Estado inicial  |             |
| Acciones        |             |
| Función sucesor |             |
| Objetivo        |             |
| Costo           |             |

## Actividad 2. Búsqueda de costo uniforme

Antes de implementar A\*, analice la búsqueda de costo uniforme.

En UCS la prioridad de un nodo está determinada únicamente por:

$$f(n) = g(n)$$

Localice en `search.py`:

```
1 def uniformCostSearch(problem):
2     "Search the node of least total cost first."
```

Para administrar la frontera puede utilizarse la estructura:

```
1 util.PriorityQueue()
```

disponible en `util.py`.

### Actividad 2 – Analizando UCS

Ejecute una búsqueda de costo uniforme sobre un laberinto del proyecto.

Registre:

| Métrica             | Resultado |
|---------------------|-----------|
| Costo del camino    |           |
| Longitud del camino |           |
| Nodos expandidos    |           |
| Tiempo              |           |

Este resultado será utilizado posteriormente como línea base para comparar A\*.

### Actividad 3. Implementación de A\*

Localice en `search.py`:

```
1 def aStarSearch(problem, heuristic=nullHeuristic):
2     "Search the node that has the lowest combined cost and heuristic first
   ."
```

La prioridad utilizada por A\* debe calcularse mediante:

$$f(n) = g(n) + h(n)$$

El siguiente pseudocódigo describe el procedimiento general:

```
frontera <- estado inicial
g(inicio) <- 0

mientras frontera no este vacia:

    n <- extraer estado con menor f(n)

    si n es objetivo:
        retornar camino
```

```

para cada sucesor de n:

    calcular nuevo costo g

    si encontramos un camino mejor:

        actualizar costo
        actualizar camino

    f(n) = g(n) + h(n)

    insertar estado en frontera

```

### Actividad 3 – Programando A\*

Complete la función:

```

1 def aStarSearch(problem, heuristic=nullHeuristic):

```

Su implementación debe:

1. utilizar una cola de prioridad;
2. comenzar desde `problem.getStartState()`;
3. utilizar `problem.isGoalState()`;
4. obtener sucesores mediante `problem.getSuccessors()`;
5. acumular el costo  $g(n)$ ;
6. calcular la prioridad  $g(n) + h(n)$ ;
7. evitar expansiones innecesarias;
8. retornar una lista de acciones.

### Actividad 4. Heurística nula

El proyecto proporciona:

```

1 def nullHeuristic(state, problem=None):
2     return 0

```

Por lo tanto:

$$h(n) = 0$$

y A\* utiliza:

$$f(n) = g(n) + 0 = g(n).$$

**Actividad 4 – A\* sin información**

Ejecute A\* utilizando `nullHeuristic`.  
Compare el resultado con UCS.

| Algoritmo        | Costo | Expandidos | Tiempo |
|------------------|-------|------------|--------|
| UCS              |       |            |        |
| $A^* + h(n) = 0$ |       |            |        |

**Pregunta de análisis**

¿Qué comportamiento observa?  
Explique por qué A\* con:

$$h(n) = 0$$

presenta un comportamiento equivalente o muy similar a búsqueda de costo uniforme.

**Actividad 5. Distancia Manhattan**

Una primera heurística para Pac-Man es la distancia Manhattan.  
Sean:

$$n = (x_n, y_n)$$

y

$$g = (x_g, y_g)$$

la posición actual y la posición objetivo.  
La distancia Manhattan se define como:

$$h_M(n) = |x_n - x_g| + |y_n - y_g|.$$

En `searchAgents.py` encontrará:

```

1 def manhattanHeuristic(position, problem, info={}):
2     xy1 = position
3     xy2 = problem.goal
4     return abs(xy1[0] - xy2[0]) + \
5         abs(xy1[1] - xy2[1])

```

**Actividad 5 – A\* con Manhattan**

Ejecute A\* utilizando la distancia Manhattan.

Observe gráficamente las celdas exploradas.

Registre:

| Algoritmo      | Costo | Expandidos | Tiempo |
|----------------|-------|------------|--------|
| UCS            |       |            |        |
| A* + Manhattan |       |            |        |

**Pregunta de análisis**

¿Por qué la distancia Manhattan puede orientar correctamente a Pac-Man aunque la heurística no considere directamente las paredes del laberinto?

**Actividad 6. Distancia Euclidiana**

Otra posibilidad consiste en utilizar la distancia Euclidiana:

$$h_E(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}.$$

El proyecto proporciona:

```

1 def euclideanHeuristic(position, problem, info={}):
2     xy1 = position
3     xy2 = problem.goal
4     return ((xy1[0] - xy2[0]) ** 2 +
5             (xy1[1] - xy2[1]) ** 2) ** 0.5

```

### Actividad 6 – Comparación de heurísticas

Ejecute A\* utilizando:

1. heurística nula;
2. distancia Manhattan;
3. distancia Euclidiana.

Complete:

| Heurística | Longitud | Costo | Expandidos | Tiempo |
|------------|----------|-------|------------|--------|
| $h(n) = 0$ |          |       |            |        |
| Manhattan  |          |       |            |        |
| Euclidiana |          |       |            |        |

### Pregunta de análisis

Pac-Man se desplaza únicamente horizontal o verticalmente.  
 ¿Cuál de las dos heurísticas representa mejor este tipo de movimiento?  
 Justifique su respuesta utilizando los resultados experimentales.

## Actividad 7. Problema de las cuatro esquinas

Hasta este momento el estado puede representarse simplemente mediante:

$$s = (x, y).$$

Ahora Pac-Man deberá visitar las cuatro esquinas del laberinto.

El proyecto proporciona:

```
1 class CornersProblem(search.SearchProblem):
```

y define las esquinas mediante:

```
1 self.corners = (  
2     (1,1),  
3     (1,top),  
4     (right,1),  
5     (right,top)  
6 )
```

Sin embargo, la posición actual ya no contiene toda la información necesaria.

Considere dos situaciones:

$$s_1 = (5, 4)$$

y

$$s_2 = (5, 4).$$

Aunque Pac-Man se encuentre en la misma posición, en  $s_1$  podría haber visitado una esquina mientras que en  $s_2$  podría haber visitado tres.

Por lo tanto, ambos estados no son equivalentes desde la perspectiva del problema.

### Actividad 7 – Diseñando el estado

Proponga una representación del estado que incluya:

- posición actual de Pac-Man;
- información acerca de las esquinas visitadas.

Una posible representación conceptual es:

$$s = (\text{posición}, \text{esquinas visitadas}).$$

Complete en `CornersProblem`:

```

1 def getStartState(self):
2     ...
3
4 def isGoalState(self, state):
5     ...
6
7 def getSuccessors(self, state):
8     ...

```

### Actividad 8. Heurística para las esquinas

El archivo inicial contiene:

```

1 def cornersHeuristic(state, problem):
2
3     corners = problem.corners
4     walls = problem.walls
5
6     return 0

```

La función:

$$h(n) = 0$$

es admisible, pero no proporciona información útil para orientar la búsqueda.

### Reto

Diseñe una función heurística que estime el costo necesario para visitar las esquinas que aún no han sido alcanzadas.

Una primera aproximación puede utilizar:

$$h(n) = \max_{c \in C_p} d_M(n, c)$$

donde  $C_p$  representa el conjunto de esquinas pendientes.

Analice posteriormente si es posible diseñar una heurística más informativa que tenga en cuenta que Pac-Man debe visitar **todas** las esquinas restantes.

Implemente:

```

1 def cornersHeuristic(state, problem):
2
3     # Su solución
4
5     return heuristicValue

```

## Admisibilidad

Una heurística es admisible cuando nunca sobreestima el costo real restante.

Formalmente:

$$0 \leq h(n) \leq h^*(n)$$

donde:

$$h^*(n)$$

representa el costo óptimo real desde  $n$  hasta un estado objetivo.

### Pregunta de análisis

Explique por qué su heurística para **CornersProblem** es admisible.

No es suficiente indicar únicamente que “la heurística funciona”.

Debe argumentar por qué el valor calculado constituye una cota inferior del costo real restante.

## Consistencia

Una heurística consistente debe satisfacer:

$$h(n) \leq c(n, n') + h(n')$$

para cualquier estado  $n$  y cualquiera de sus sucesores  $n'$ .

Además:

$$h(goal) = 0.$$

### Pregunta de análisis

Analice si la heurística propuesta para el problema de las esquinas puede considerarse consistente.

Justifique su respuesta.

## Actividad 9. Experimento comparativo

Ejecute el problema utilizando diferentes estrategias.  
Complete la siguiente tabla:

| Método                       | Costo | Expandidos | Tiempo | Óptimo |
|------------------------------|-------|------------|--------|--------|
| UCS                          |       |            |        |        |
| $A^* + h = 0$                |       |            |        |        |
| $A^* +$ heurística básica    |       |            |        |        |
| $A^* +$ heurística propuesta |       |            |        |        |

Calcule el factor de reducción de expansiones:

$$R = \frac{N_{\text{UCS}}}{N_{A^*}}.$$

Por ejemplo, si:

$$N_{\text{UCS}} = 4000$$

y:

$$N_{A^*} = 1000,$$

entonces:

$$R = 4.$$

Esto significa que UCS expandió aproximadamente cuatro veces más estados.

## Actividad 10. Búsqueda de todos los alimentos

El siguiente nivel del problema consiste en encontrar un camino que permita recoger todos los alimentos.

El proyecto define:

```
1 class FoodSearchProblem:
```

El estado utilizado es:

```
1 (pacmanPosition, foodGrid)
```

Por lo tanto:

$$s = (p, F)$$

donde:

- $p$  representa la posición de Pac-Man;

- $F$  representa los alimentos restantes.

El objetivo se alcanza cuando:

$$|F| = 0.$$

En el código esto corresponde a:

```
1 def isGoalState(self, state):
2     return state[1].count() == 0
```

Observe que el espacio de estados aumenta considerablemente.

Si existen  $F$  alimentos, cada alimento puede encontrarse en dos condiciones:

{presente, consumido}.

Por lo tanto, conceptualmente el número de configuraciones puede crecer como:

$$2^F.$$

## Actividad 11. Diseñando foodHeuristic

Localice:

```
1 def foodHeuristic(state, problem):
2
3     position, foodGrid = state
```

Diseñe una heurística para estimar el costo necesario para consumir todos los alimentos restantes.

Una posibilidad inicial consiste en considerar el alimento más lejano:

$$h(n) = \max_{f \in F} d_M(n, f).$$

### Actividad 11 – Heurística para alimentos

Implemente una primera versión de `foodHeuristic`.

Posteriormente diseñe una segunda versión que proporcione más información sin perder las propiedades necesarias para A\*.

Compare:

| Heurística   | Costo | Expandidos | Tiempo |
|--------------|-------|------------|--------|
| $h(n) = 0$   |       |            |        |
| Heurística 1 |       |            |        |
| Heurística 2 |       |            |        |

## Optimización de la heurística

El problema proporciona el diccionario:

```
1 problem.heuristicInfo
```

que puede utilizarse para almacenar información calculada previamente.

Por ejemplo:

```
1 problem.heuristicInfo['dato'] = valor
```

Antes de repetir un cálculo puede verificarse:

```
1 if 'dato' in problem.heuristicInfo:
2     valor = problem.heuristicInfo['dato']
```

### Reto

Identifique algún cálculo realizado repetidamente por su heurística.

Utilice `problem.heuristicInfo` como mecanismo de memoria caché y compare nuevamente el tiempo de ejecución.

## Competencia de heurísticas

Cada grupo deberá diseñar una heurística propia para uno de los siguientes problemas:

- `CornersProblem`;
- `FoodSearchProblem`.

La heurística debe intentar minimizar:

$$N_{\text{expandidos}}$$

manteniendo la optimalidad de la solución.

Cada grupo deberá documentar:

1. nombre de la heurística;
2. idea utilizada;
3. representación matemática;
4. implementación;
5. justificación de admisibilidad;
6. análisis de consistencia;
7. costo obtenido;
8. número de nodos expandidos;
9. tiempo de ejecución.

## Análisis final

Responda las siguientes preguntas.

1. ¿Cuál es la diferencia fundamental entre UCS y A\*?
2. ¿Qué función cumple  $g(n)$ ?
3. ¿Qué función cumple  $h(n)$ ?
4. ¿Qué ocurre cuando  $h(n) = 0$ ?
5. ¿Cuál presentó mejores resultados: Manhattan o Euclidiana?
6. ¿Una heurística más grande siempre es mejor? Justifique.
7. ¿Por qué una heurística que sobreestima puede ser problemática?
8. ¿Por qué el estado de `CornersProblem` necesita almacenar más información que únicamente  $(x, y)$ ?
9. ¿Por qué `FoodSearchProblem` presenta un espacio de estados considerablemente mayor?

10. ¿Qué relación encontró entre la calidad de la heurística y el número de nodos expandidos?

## Resultados generales

Consolide los principales resultados experimentales.

| Experimento            | Costo | Expandidos | Tiempo |
|------------------------|-------|------------|--------|
| UCS                    |       |            |        |
| A* + heurística nula   |       |            |        |
| A* + Manhattan         |       |            |        |
| A* + Euclidiana        |       |            |        |
| A* + Corners Heuristic |       |            |        |
| A* + Food Heuristic    |       |            |        |

## Entregable

El trabajo deberá entregarse en un archivo comprimido:

GrupoXX\_Pacman\_AStar.zip

```
|
|-- search.py
|-- searchAgents.py
|-- resultados.csv
|
'-- informe.pdf
```

El informe deberá contener:

1. descripción breve del problema;
2. explicación del algoritmo A\*;
3. implementación realizada;
4. descripción de las heurísticas;
5. resultados experimentales;
6. tablas comparativas;
7. análisis de nodos expandidos;
8. discusión de admisibilidad y consistencia;
9. conclusiones.

## Conclusión

El objetivo del laboratorio no consiste únicamente en encontrar una ruta para Pac-Man.

El aspecto fundamental es comprender que una heurística introduce **conocimiento del problema** dentro del proceso de búsqueda.

Una buena heurística puede reducir significativamente el número de estados que deben explorarse sin sacrificar la calidad de la solución.

La comparación entre:

UCS       $A^* + h(n) = 0$        $A^* + \text{Manhattan}$        $A^* + \text{heurística diseñada}$

permite observar experimentalmente una de las ideas centrales de la búsqueda informada:

Mejor información  $\implies$  menor exploración del espacio de estados

siempre que la función heurística preserve las propiedades requeridas para garantizar la optimalidad.

## Referencia

Stanford University. *CS221: Artificial Intelligence — Pac-Man Assignment*.

<https://stanford-cs221.github.io/spring2023/assignments/pacman/index.html>